

Natural Language Processing (NLP) - Usecases

- Language translation
- Question Answering
- Sentiment Analysis
- Text Search (keyword and semantic)
- Text classifications
- Named Entity Recognition (NER)
- Summarization
- Paraphrase
- Topic Modelling
- Emotion recognition
- Document ranking

Natural Language Processing (NLP)

Linguistic Basics

1. Lemmatization
2. Stemming
3. Part of Speech Tagging

Text Identification/ Extraction

4. Stop Words
5. Pattern Matching
6. Sentence Segmentation
7. Named Entity Recognition

Text Representation

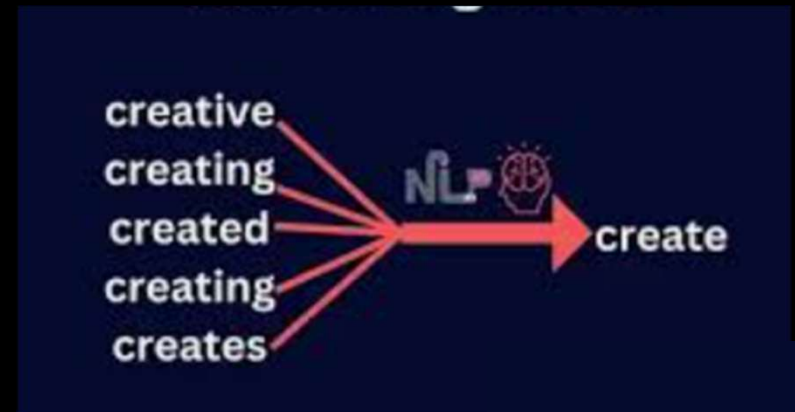
8. Tokenization
9. Word Embedding
10. Bag-of-words/TF-IDF

NLP - Stemming

Obtain the base or root form of words by removing letters from the word's end

```
from nltk.stem.porter import *  
  
p_stemmer = PorterStemmer()  
words = ["runner", "running", "ran"]  
for word in words:  
    print(word+' --> '+p_stemmer.stem(word))
```

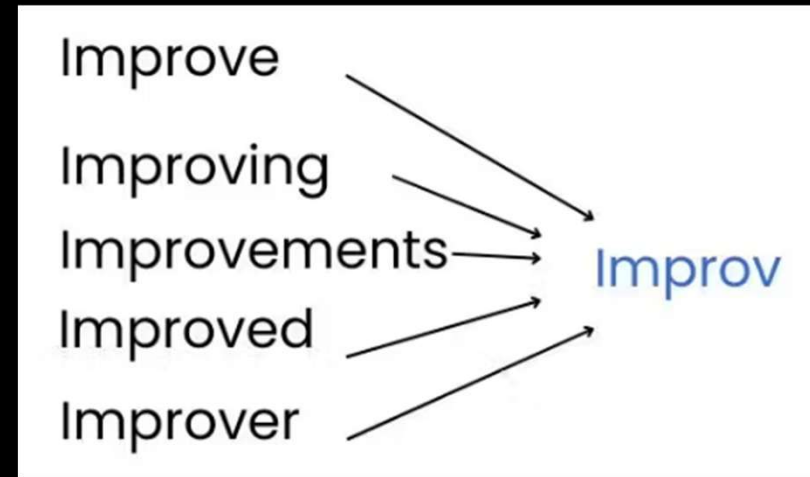
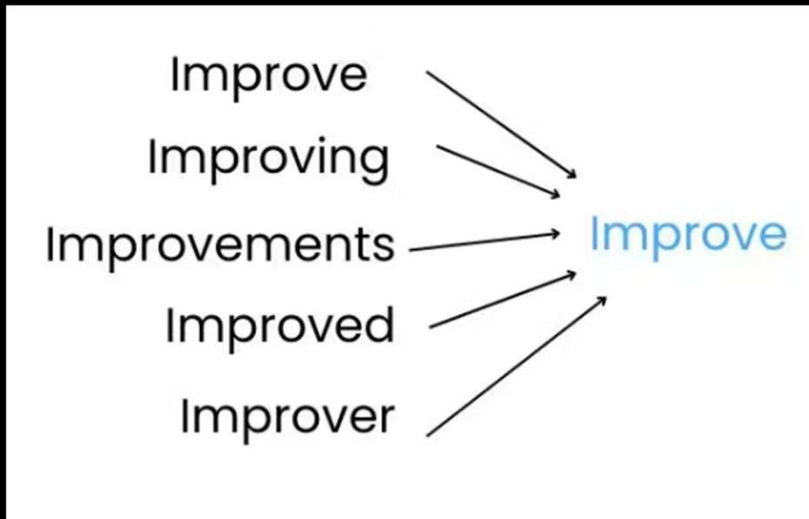
```
runner --> runner  
running --> run  
ran --> ran
```



NLP - Lemmatization

lemmatization is a more sophisticated linguistic process that aims to reduce a word to its base or dictionary form

Lemmatization takes into account factors such as part-of-speech (POS) tags and contextual understanding to ensure accurate and meaningful transformations.



Vs Stemming

Parts Of Speech (POS) Tagging

Part of speech refers to the grammatical category of a word in a sentence, such as noun, verb, adjective, adverb, pronoun, preposition, conjunction, or interjection.

Used to identify named entities and speech recognition

Stop words

Common words such as “a” and “the” occur so frequently in text that they often do not carry significant meaning compared to nouns, verbs, and modifiers.

Spacy provides a built-in list of approximately 305 English stop words that can be readily utilized.

Text representation - tokenization

Tokenization involves breaking down the original text into smaller components known as tokens. These tokens can be created based on contiguous sequences of characters or words.

N-gram refers to a consecutive sequence of n items, where an item can be a character or a word.

<https://platform.openai.com/tokenizer>

Determines the vocabulary of the model

Text representation – TF-IDF

Term Frequency (TF) / Inverse Document Frequency (IDF)

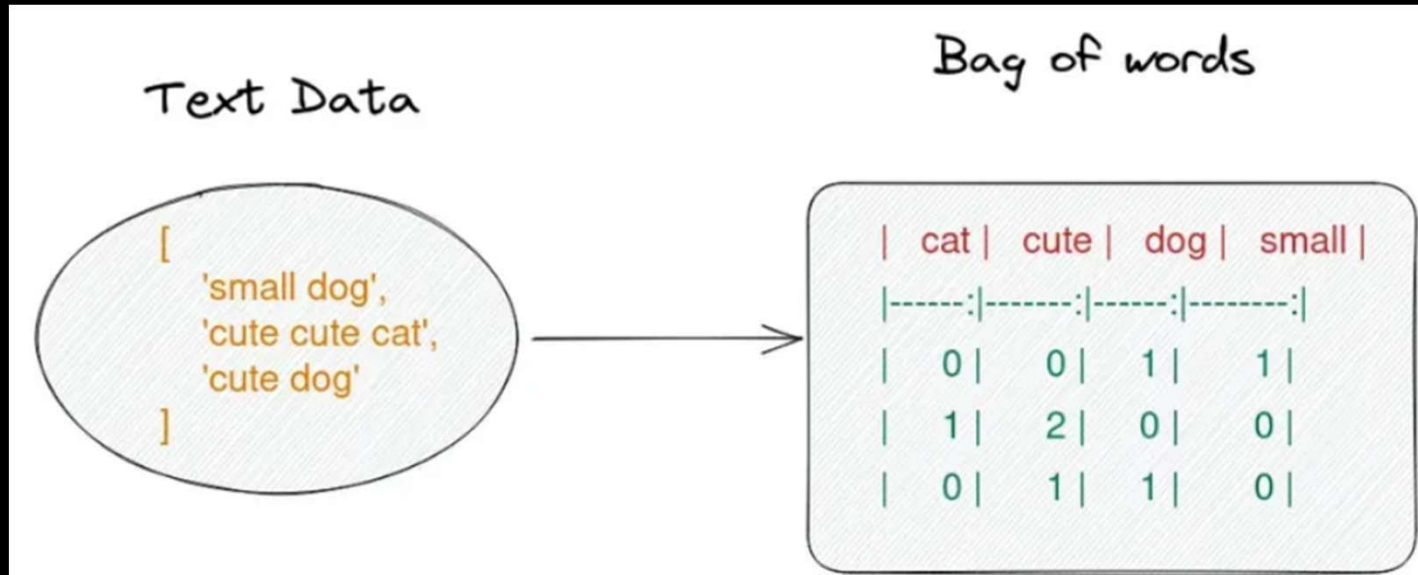
Term Frequency (TF)

This measures how frequently a term (word) appears in a document. It assigns a higher weight to words that occur more frequently within the document.

Inverse Document Frequency (IDF)

This part measures how unique or rare a term is across all documents. It assigns a higher weight to words that appear less frequently across the corpus but provide more unique or informative content within a document.

Text representation – Bag of Words



Converts each word into a vector based on the frequency of the text, without considering order

Text representation – TF-IDF

Term Frequency (TF) / Inverse Document Frequency (IDF)

Corpus D		
$+1 \rightarrow$	d_1 A quick brown fox jumps over the lazy dog. What a fox!	$TF-IDF = 0.17 \times 0 = 0$ (“fox”, d_1 , D)
$+1 \rightarrow$	d_2 A quick brown fox jumps over the lazy fox. What a fox!	$TF-IDF = 0.25 \times 0 = 0$ (“fox”, d_2 , D)

Answer: Using TF-IDF, the word “fox” is equally relevant for both document d_1 and document d_2

Question: How word fox is relevant to corpus D documents?

Solution:

TF-IDF

TF is the frequency of any “term” in a given “document”.

IDF is constant per corpus, and accounts for the ratio of documents that include that specific “term”.

$$TF(\text{“fox”}, d_1) = 2 / 12 = 0.17$$

$$TF(\text{“fox”}, d_2) = 3 / 12 = 0.25$$

$$IDF(\text{“fox”}, D) = \log(2/2) = 0$$

Text representation – TF-IDF

Document d1 (Cardiology report)

The patient shows elevated troponin levels indicating myocardial infarction and cardiac injury.

Document d2 (General infection report)

The patient has fever, cough, and signs of viral infection with mild inflammation.

$TF(\text{troponin}, d1) \rightarrow \text{non-zero}$ (appears once in a short report $\rightarrow$ relatively important)

$TF(\text{troponin}, d2) \rightarrow 0$

$IDF(\text{troponin}) = \log(2/1) > 0$

TF-IDF

d1: $TF \times IDF \rightarrow \text{high score}$

d2: $TF \times IDF \rightarrow 0$

TF-IDF – Real world applications

1. Clinical document classification

Cardiology vs Infectious disease vs Oncology

2. Triage systems

Detect **critical cardiac cases** using terms like:

troponin

infarction

arrhythmia

3. Medical search systems

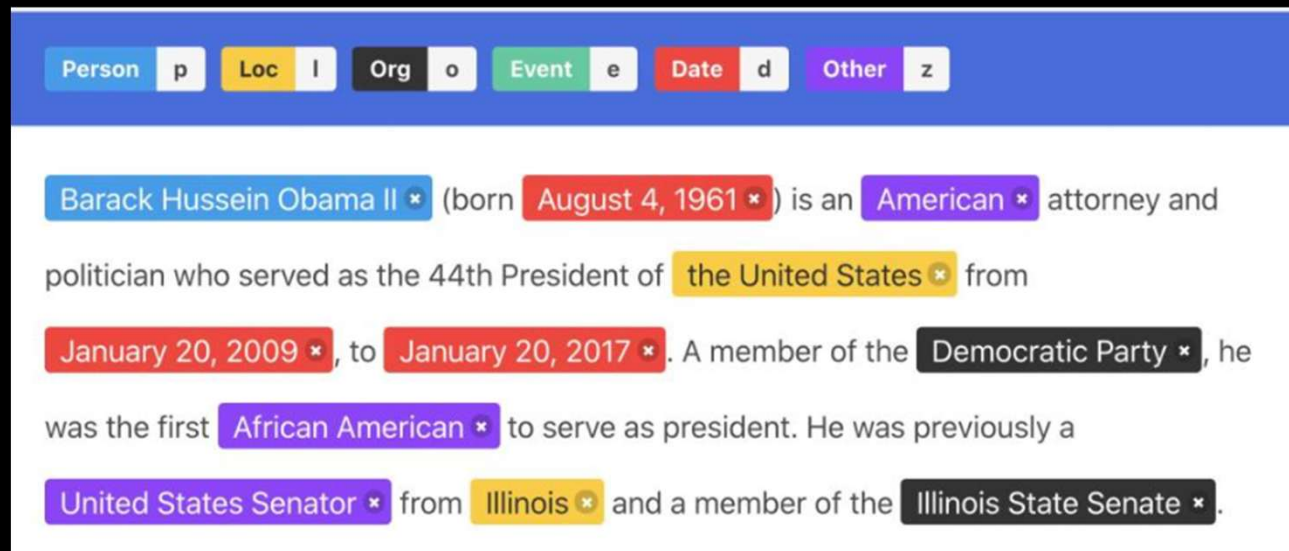
Query: *“elevated troponin”*

Retrieves cardiac reports, not general infection notes

Named Entity Recognition (NER)

Named Entity Recognition (NER) entails the identification and classification of named entities present in the given text.

Named entities represent recognizable entities : individuals, organizations, locations, dates, numerical expressions, and others.



The screenshot displays a web-based NER interface. At the top, a blue header bar contains seven labels with corresponding single-letter codes: 'Person' (p), 'Loc' (l), 'Org' (o), 'Event' (e), 'Date' (d), and 'Other' (z). Below this, a text snippet about Barack Obama is shown. The text is: 'Barack Hussein Obama II (born August 4, 1961) is an American attorney and politician who served as the 44th President of the United States from January 20, 2009, to January 20, 2017. A member of the Democratic Party, he was the first African American to serve as president. He was previously a United States Senator from Illinois and a member of the Illinois State Senate.' Each entity in the text is enclosed in a colored box with a small 'x' icon in the top right corner. The labels used are: 'Person' (blue), 'Date' (red), 'Other' (purple), 'Loc' (yellow), 'Org' (dark grey), and 'Event' (green).

Entity	Label	Code
Barack Hussein Obama II	Person	p
August 4, 1961	Date	d
American	Other	z
the United States	Loc	l
January 20, 2009	Date	d
January 20, 2017	Date	d
Democratic Party	Org	o
African American	Other	z
United States Senator	Other	z
Illinois	Loc	l
Illinois State Senate	Org	o

<https://demos.explosion.ai/displacy-ent>

Topic Modeling (LDA/genism)

LDA

LDA (Latent Dirichlet Allocation) is a probabilistic topic modeling technique that discovers hidden topics in a collection of documents and represents each document as a mixture of those topics.

For business users

LDA is an unsupervised technique that automatically discovers common themes or topics across a set of documents.

For data scientists

LDA is a generative probabilistic model that assumes documents are mixtures of latent topics and topics are distributions over words.

LDA is a generative model

Generative Models

Learn:

$$P(X, Y)$$

or

$$P(\text{Words, Topics})$$

Examples:

- LDA
- Naive Bayes
- Hidden Markov Models

They model how data is produced.

Discriminative Models

Learn:

$$P(Y|X)$$

Examples:

- Logistic Regression
- SVM
- Neural classifiers

They focus on predicting labels.

Limitations of BoW and TF/IDF

BoW and TF-IDF are traditional text representation techniques, they have major limitations:

No Semantic Meaning, cannot capture similarity

Represent text as sparse vectors where each word is a separate dimension.

For example, "king" and "queen" have no relationship in BoW/TF-IDF.

High Dimensionality (Sparse Representation)

If a vocabulary has 100,000 words, each document is a 100,000-dimensional vector, making computations inefficient.

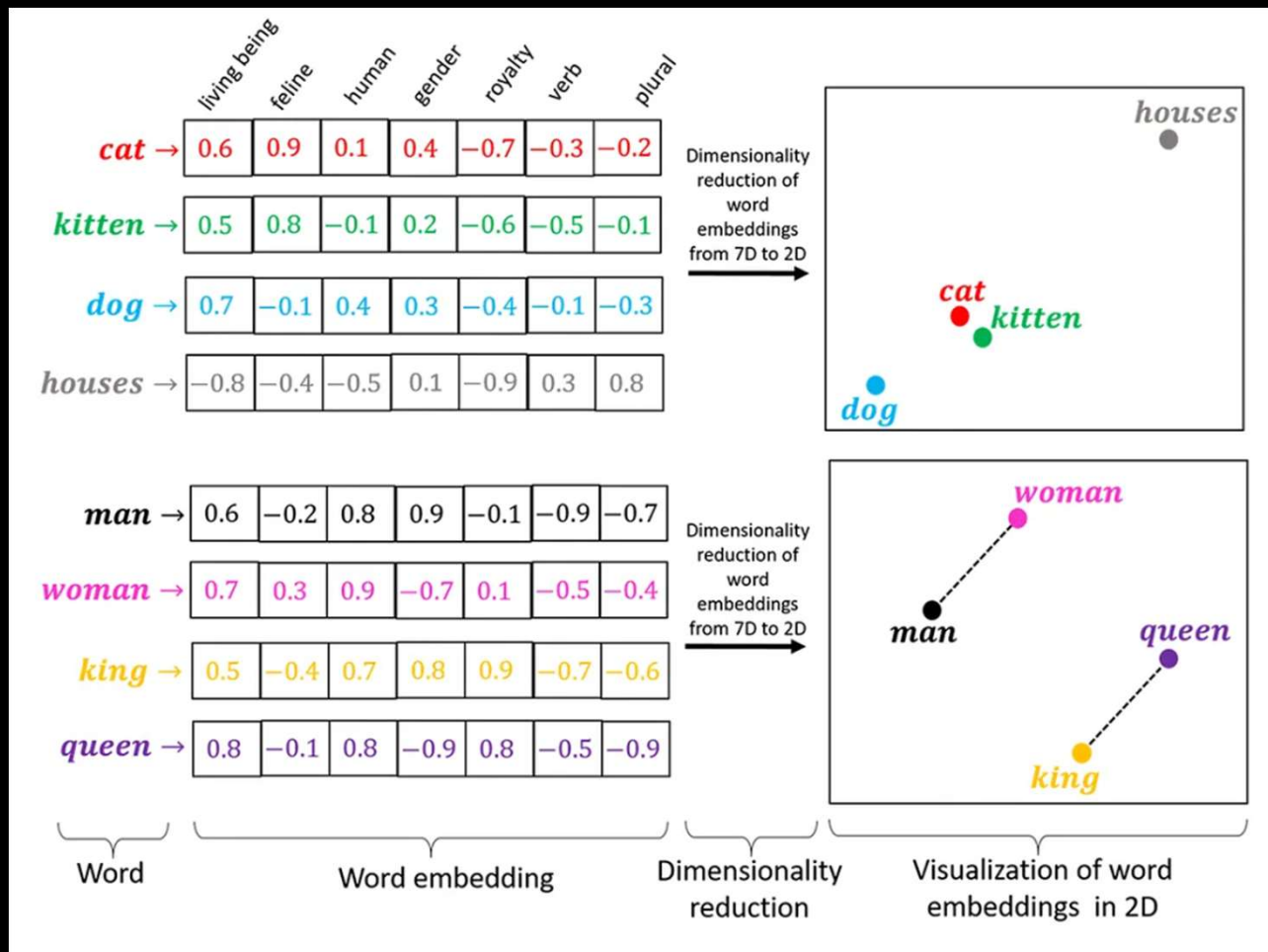
No Context Awareness

BoW & TF-IDF treat all occurrences of a word the same, regardless of sentence structure.

No Word Order Information

BoW and TF-IDF ignore word order, so "I love cats" and "Cats love me" have the same representation.

Vectors and Embeddings



0.6 =
00011001 11000101

16bit
8bit = 0 to 2^8
4 bit: 0 to 16

Video: Vectors and Embeddings

Word2vec (Google 2013)

Word2Vec is a

- Neural network-based model for learning word embeddings
- Introduces word embeddings: dense vector representations of words that capture their semantic meaning based on the context they appear in.
- It transforms words into high-dimensional numerical vectors, allowing words with similar meanings to have similar vector representations.

A major breakthrough in NLP

Word2vec key features

Dense Vector Representations

Represents words as low-dimensional dense vectors (e.g., 300 dimensions instead of 100,000).

Captures Semantic Meaning

Similar words have similar vector representations:

king - man + woman $\approx$ queen

Learns Context from Word Usage

Uses CBOW (Continuous Bag of Words) and Skip-gram to learn relationships based on nearby words.

Handles Synonyms and Similarity

Places similar words close in vector space, making it useful for NLP tasks.

How Word2vec learns Symantec meaning and relationship?

Self supervised learning

Skip-gram Model

Given a word, it predicts surrounding words.

Example: From "The cat sits on the mat," the model learns that "cat" is often surrounded by words like "sits," "on," and "mat."

This helps capture the semantic meaning of "cat."

Continuous Bag of Words (CBOW) Model

Given surrounding words, it predicts the missing word.

Example: Given "The ____ sits on the mat," the model predicts "cat."

Embedding model training process

Corpus snippet:

"A furry cat sat on the mat."

"A soft bear toy is popular with kids."

...

Training step:

Input word	Desired context	Effect on vectors

cat	furry, sat, mat	cat $\leftrightarrow$ furry pulled closer
bear	soft, toy, kids	bear $\leftrightarrow$ soft pulled closer

After many steps:

`cat`, `dog`, `bear` cluster near the "fur/softness" direction;

`soft` and `fluffy` also align along that ray;

but no single coordinate equals "texture." It's the **direction** created by many coordinates acting together.

Output of the training process – embedding weights

The output of the training process are **embedding weights**, the numbers in the matrix of $\#words \times \#dimensions$

For example, for a corpus of 50000 words and 768 dimensions, the matrix of 50Kx768 cells values will be adjusted during training

The result is 50Kx768 (~38Million) adjusted embedding weights

Note:

- word2vec is not a pre-trained model (popular pre-trained models based on it are available)
- It used words directly as tokens (not sub-words). The vocabulary can explode based on training data.

Contextual Embeddings

Static embedding models (such as word2vec) have a problem:

The embedding for word bank will be same in both sentences below

*I went to the **bank** to deposit money*

*I sat on the **bank** of river Ganga*

Contextual Embeddings

- The static embeddings from training become the starting point
We stop calling it **weight** and start calling it an Activation (or "Hidden State").
It is now Data that can be manipulated.
- Embeddings will be nudged toward the correct geometry based on context
The attention layer and FFNN layers will transform it based on the attention and FFNN weights (learnt during training) to adjust the vector, based on the context (move it toward finance/nature in this example)

Note: Weights are always frozen once trained. They cannot be adjusted during inferencing

Popular embedding models

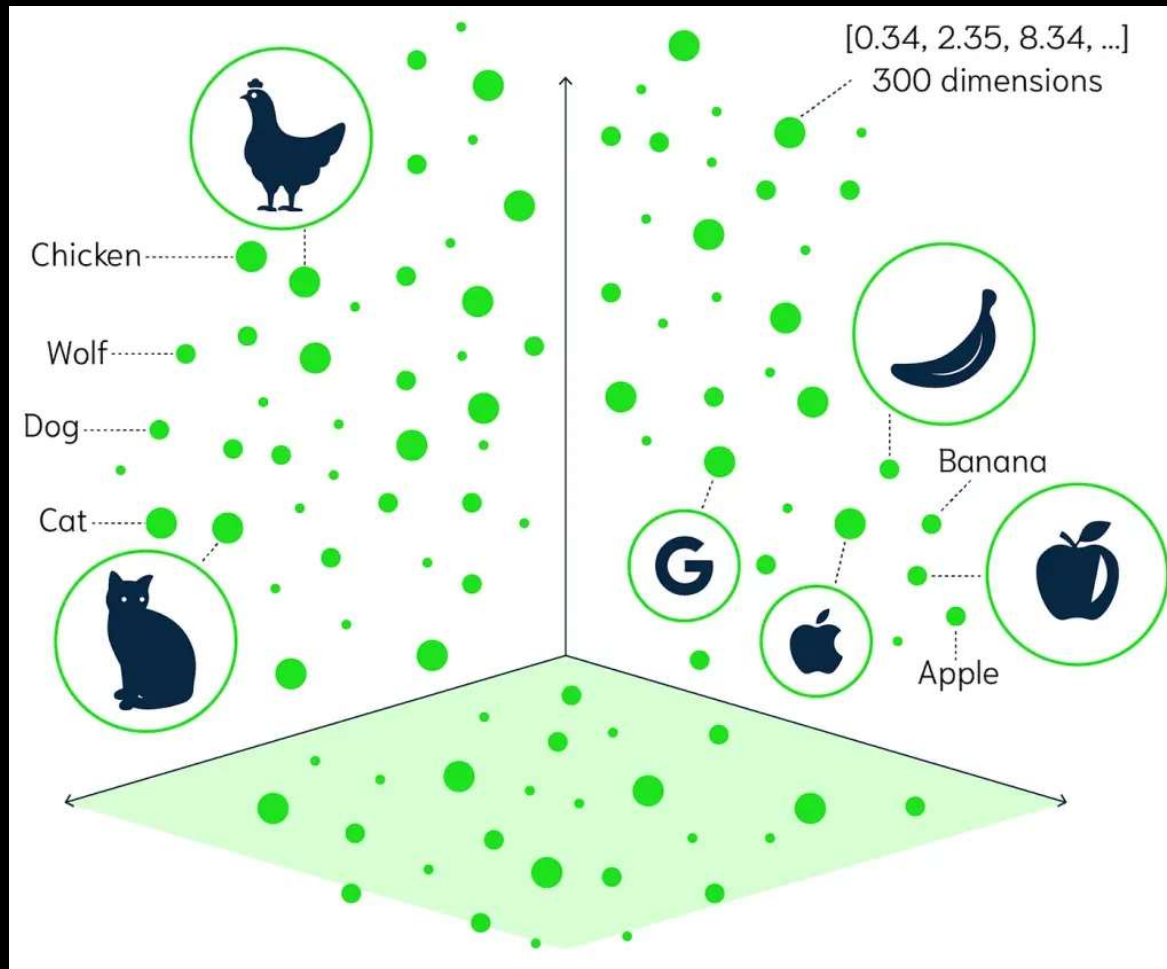
Legacy Static embedding models:

- Word2vec (Google news), GloVe (Wikipedia+Gigaword)

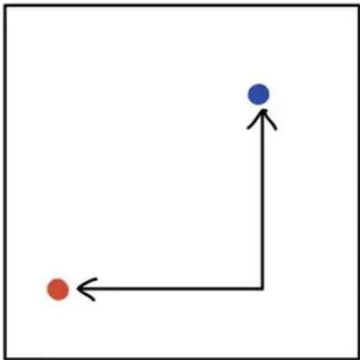
Modern contextual embedding models:

- BERT
- SBERT (Sentence transformer)
(popular for RAG applications)
Variants:
 - all-MiniLM-L6-v2 (very fast, widely used)
 - mpnet-base-v2 (high accuracy)
 - **bge-* family (current SOTA)**
- OpenAI Embedding Models
 - text-embedding-3-small
 - text-embedding-3-large
 - text-embedding-ada-002 (older)
Used heavily in production RAG systems
- Domain specific embedding models
 - MedBERT

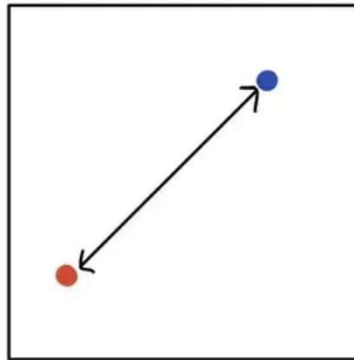
Similarity Search with embeddings



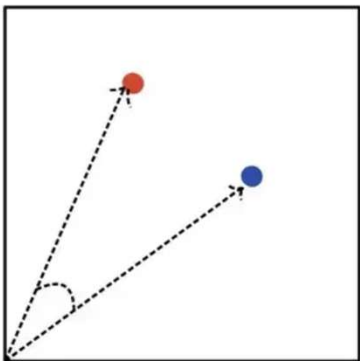
Measuring similarity – distance metrics



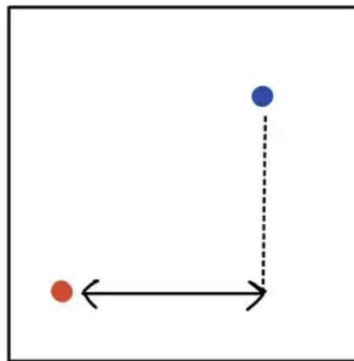
Manhattan



Euclidean



Cosine



Chebyshev

Choosing the right distance metric depends on the specific characteristics and requirements of the application.

Cosine similarity is popular

More details:

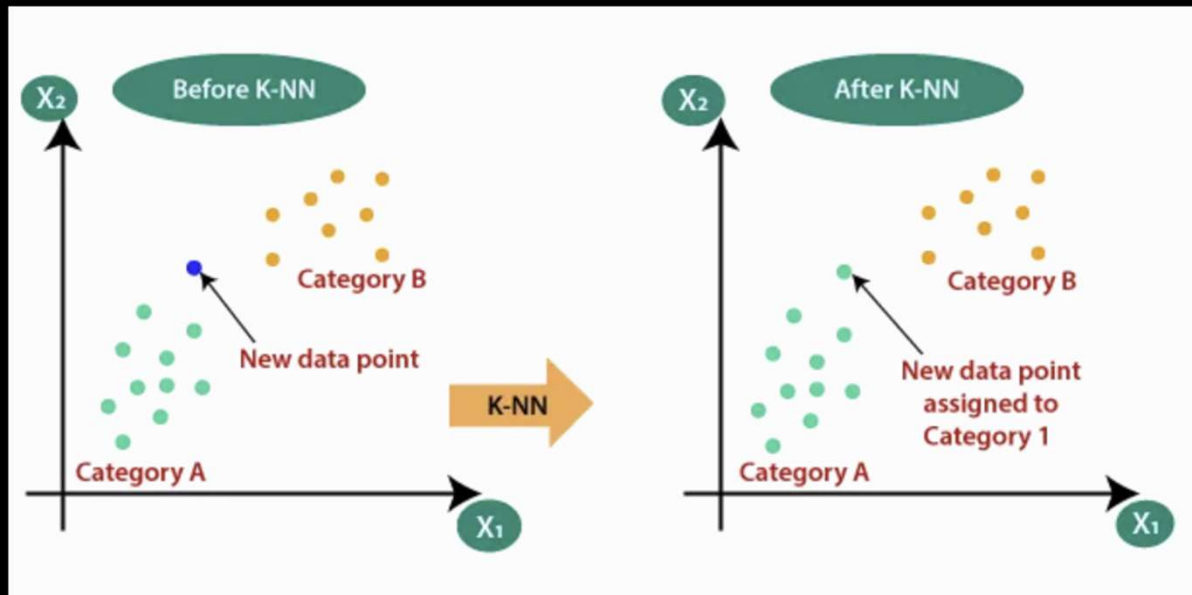
<https://www.truefoundry.com/blog/similarity-search>

Performing similarity – kNN

K-Nearest Neighbors (k-NN)

K-Nearest Neighbors (k-NN) is a popular algorithm used to find the closest vectors to a given query vector.

Disadvantages: Computationally expensive for large datasets since it involves calculating the distance to every vector.



Performing similarity – Approximate Nearest Neighbor (ANN)

To address the inefficiency of k-NN with large datasets, Approximate Nearest Neighbor (ANN) methods provide a faster, albeit less precise, alternative.

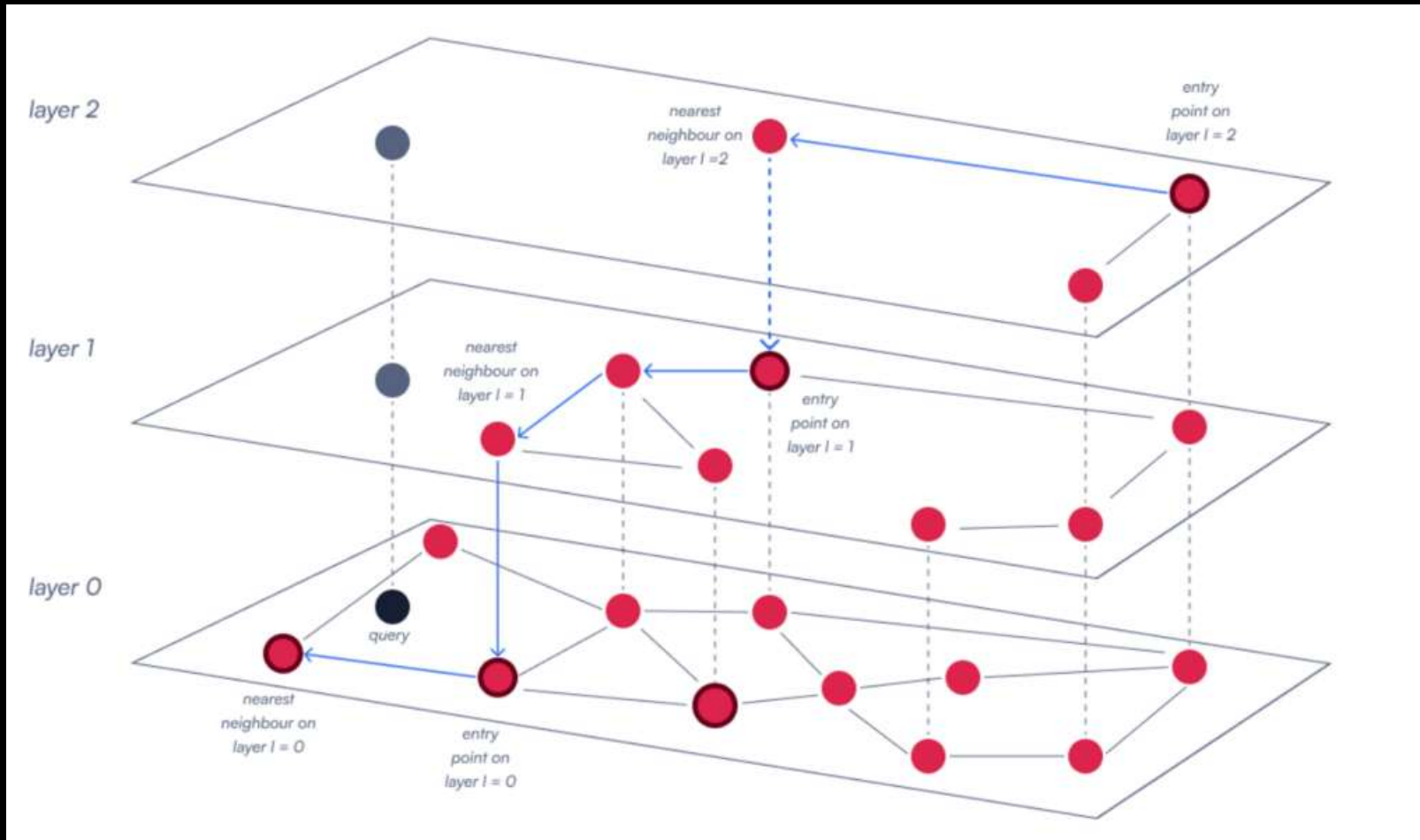
ANN algorithms aim to find a “good guess” of the nearest neighbors, trading off some accuracy for speed.

Indexing Techniques: ANN algorithms use indexing structures like KD-Trees, Ball Trees, and VP-Trees, HNSW to partition the vector space and narrow down the search area.

It also uses other optimization techniques such as hashing, clustering

FAISS (Facebook AI Similarity Search) library offers various APIs to do similarity search

Hierarchical Navigable Small World (HNSW)



<https://qdrant.tech/course/essentials/day-2/what-is-hnsw/>

Combining lexical search (BM25) with semantic – hybrid search

Lexical search (keyword based) performs well for exact match:

- Patient id
- Icd-code
- Any other coded text

Semantic Search (semantic similarity) performs well when we want to search on meaning

- Search for “chest pain” should show up pneumonia, myocardial infraction etc.

Depending on use-case, a hybrid search approach maybe optimal

The results of hybrid search will be fed to a reranker such as RRF

Typical Search Pipeline

```
Raw doctor query
↓
Spell correction using clinical vocabulary
↓
Synonym / abbreviation expansion
↓
Boolean parsing
↓
BM25 retrieval + semantic retrieval
↓
Hard Boolean filtering
↓
Reranking
↓
Evidence highlighting
```

Example

Original query:

pnemonia AND ibuprofin

Corrected query:

pneumonia AND ibuprofen

Expanded query:

(pneumonia OR CAP OR LRTI OR chest infection)

AND

(ibuprofen OR Advil OR Brufen OR NSAID)

- Fuzzy matching for spelling correction
- Clinical dictionary expansion can greatly improve BM25 results

How does BGE differ from BERT?

- Architecture-wise
 - Both use transformers
 - Both use attention
 - Both use similar parameter counts

The difference is:

- Pooling strategy
- Training signal
- Contrastive learning setup

Contrastive learning

Query: "open a savings account"

Positive doc: "banks offer savings accounts"

Negative doc: "river bank erosion"

Objective:

SCSS

$\cos(\text{query}, \text{positive}) \gg \cos(\text{query}, \text{negative})$

This directly optimizes embedding geometry.